



Вычислительная геометрия и алгоритмы компьютерной графики

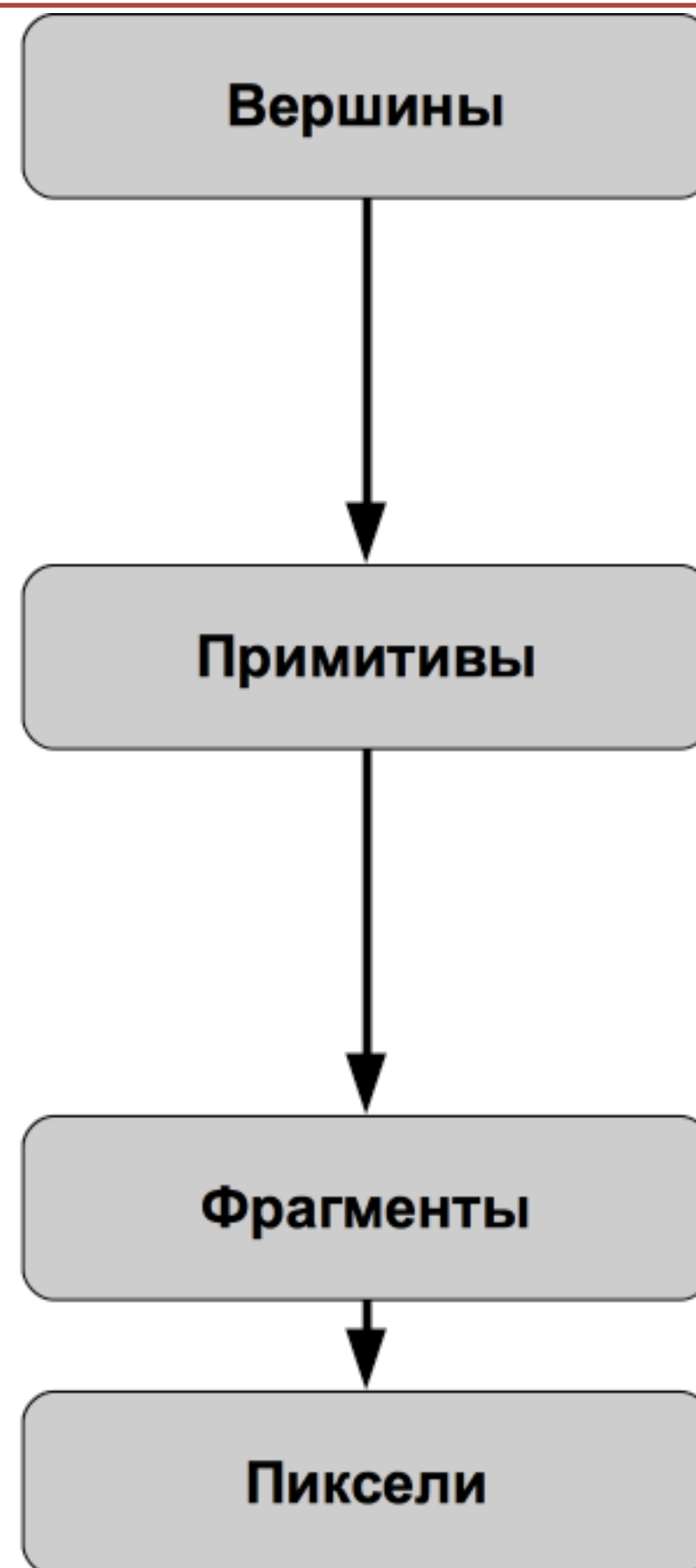
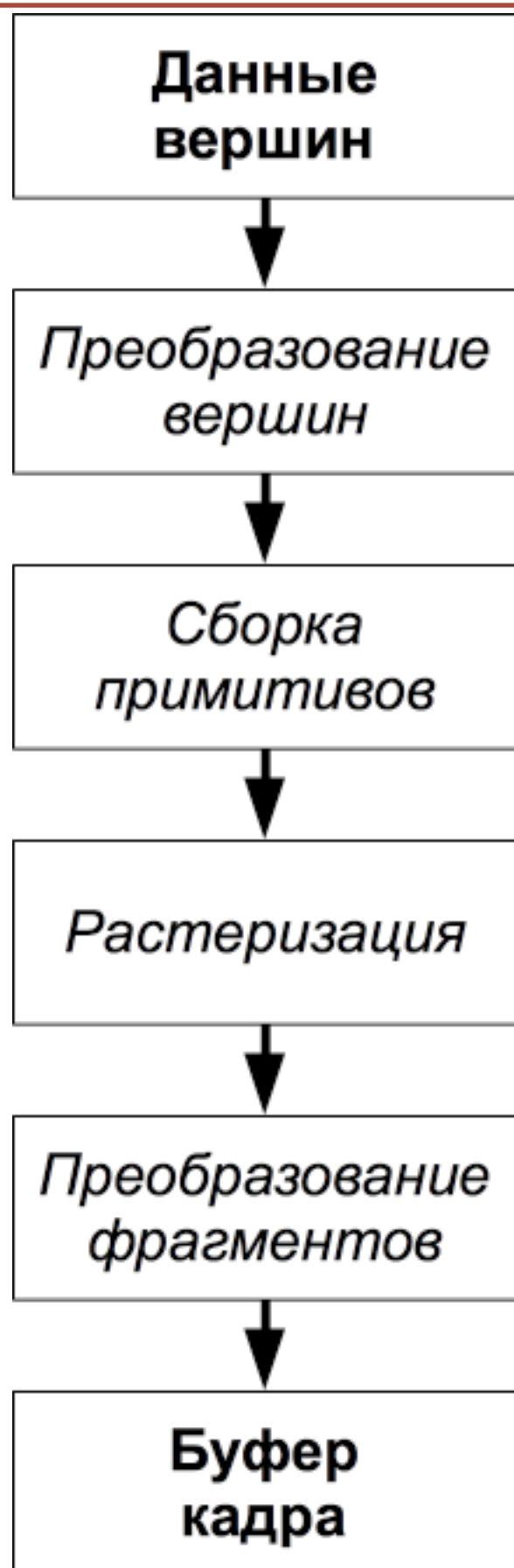
Лекция №4: Шейдеры. Язык GLSL

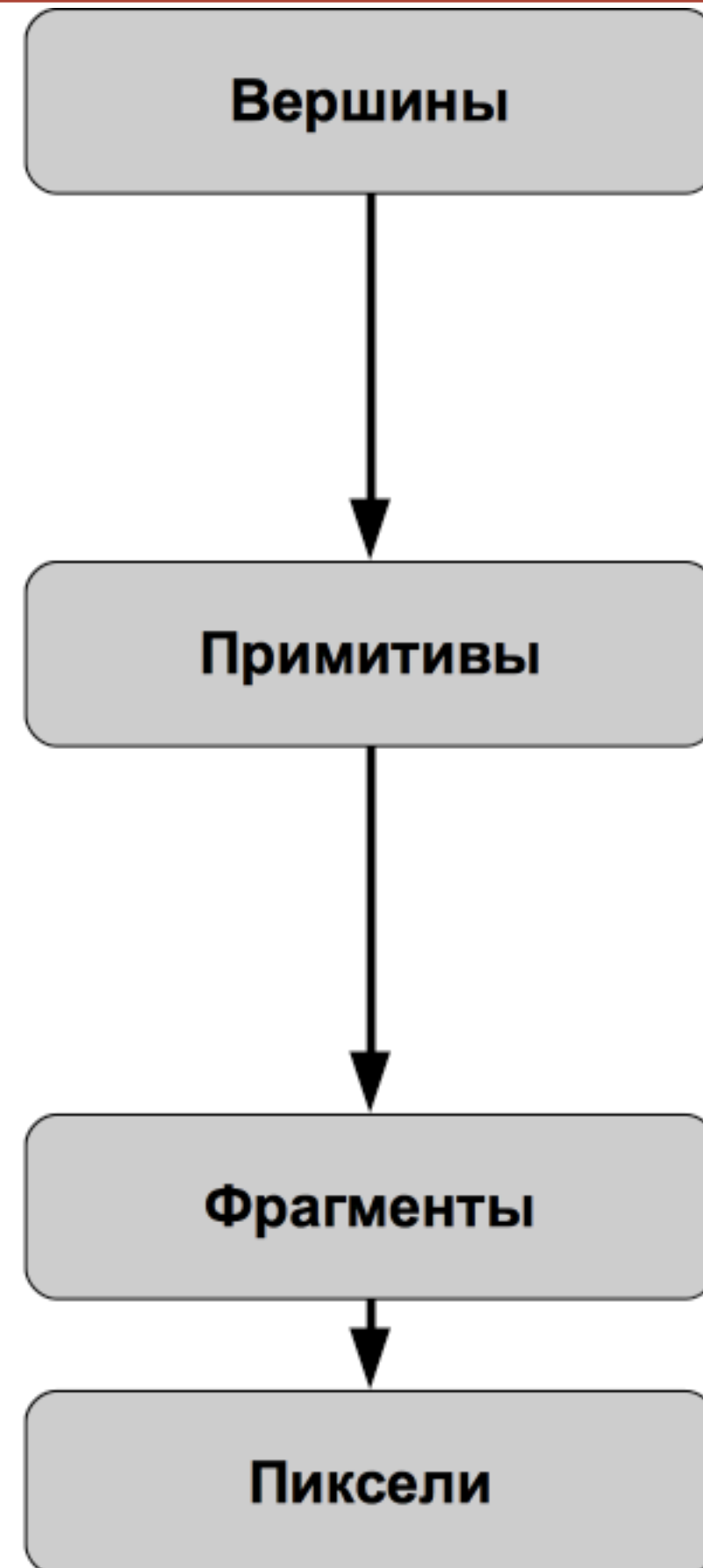
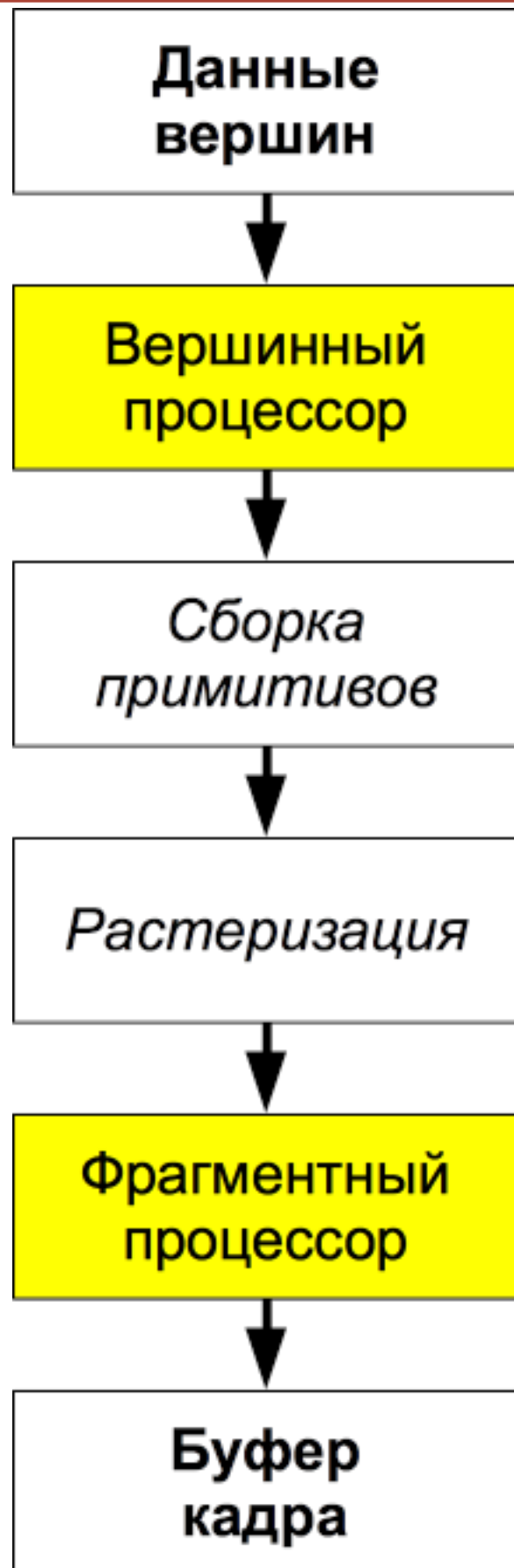
к.ф.-м.н., доцент

Рябинин Константин Валентинович,
kostya.ryabinin@gmail.com

ст. преподаватель

Лабутин Иван Александрович,
linuxrf@gmail.com





Шейдер – это микропрограмма для одной из ступеней графического конвейера, используемая для определения окончательных параметров объекта или изображения

Свойства:

- Автономность
- Аппаратная поддержка
- Специализированный язык программирования
- Параллелизм по данным
- Узкая специализация

Назначение шейдеров:

- **Создание визуальных эффектов**
- **Превращение графического конвейера из неуправляемого в управляемый**
- **Значительное увеличение свободы управления результатом визуализации**
- **Унификация механизма создания визуальных эффектов любой сложности**

Достоинства шейдеров:

- **Очень высокая эффективность**
- **Свобода создания визуальных эффектов**
- **Децентрализация кода**
- **Межпроектное переиспользование**

- 1. Загрузка из файла (или из строковой константы)**
- 2. Компиляция («на лету», во время выполнения основной программы)**
(из исходного или SPIR-V кода)
- 3. Встраивание в конвейер («активация»)**
- 4. Множественное выполнение (для каждой обрабатываемой данной ступенью конвейера сущности за один рендеринг шейдер выполняется ровно один раз)**
- 5. Отсоединение от конвейера («деактивация»)**
- 6. Удаление из памяти**

1. **Вершинный шейдер** – микропрограмма, обрабатывающая за раз **одну вершину объекта**

Задача: определить положение и прочие характеристики вершины

2. **Фрагментный шейдер** – микропрограмма, обрабатывающая за раз **один фрагмент изображения**

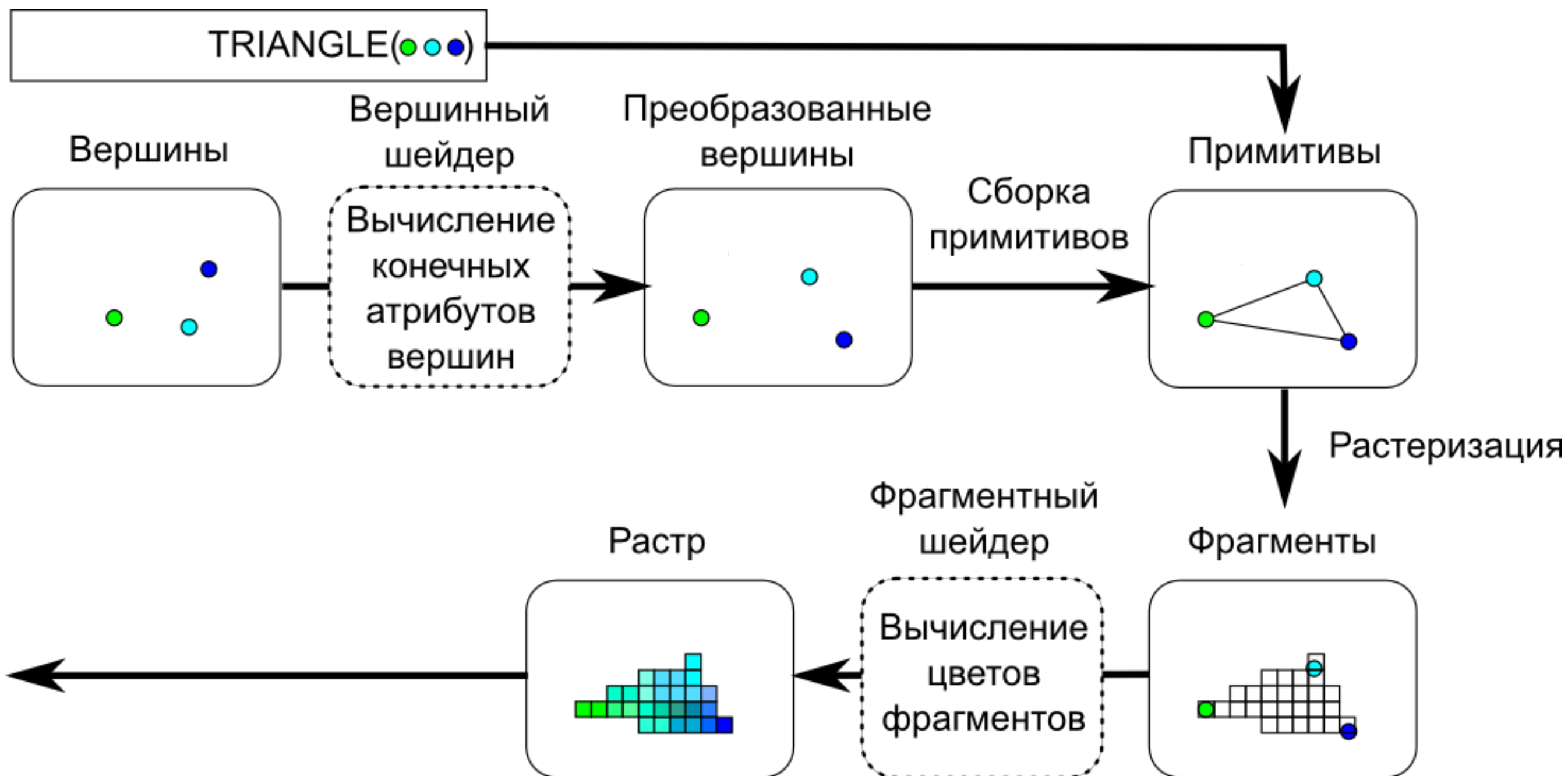
Задача: определить цвет и прочие характеристики фрагмента

3. Геометрический шейдер

4. Тесселяционный шейдер

5. Вычислительный шейдер

- Поверхности в OpenGL хранятся в виде массивов вершин (каждая вершина – набор атрибутов типа float)
- Тип связности вершин (способ объединения в примитивы) указывается уже при рендеринге, в качестве параметра вызова отрисовки
- Массив вершин может храниться как в оперативной памяти, так и в видеопамяти.
Второй способ предпочтительнее по скорости
- Для хранения вершин в видеопамяти используется специальная структура - **вершинный буфер** (vertex buffer)
- Часто кроме буфера вершин используется ещё и **буфер индексов** (index buffer), что позволяет устранить дублирование информации о вершинах
- Работа с вершинными буферами осуществляется при помощи API-функций OpenGL



Шейдер – общее название для семейства специализированных микропрограмм

Для написания шейдеров используются специализированные языки программирования, характеризующиеся:

- **Процедурной парадигмой**
- **Тьюринг-полнотой**
- **Наличием специализированных типов данных и встроенных функций для работы с обрабатываемыми сущностями**
- **Как правило, C-подобным синтаксисом**

- **GLSL (Graphics Library Shader Language)**
Шейдерный язык от ARB (OpenGL, Vulkan)
- **HLSL (High-Level Shader Language)**
Шейдерный язык от Microsoft
- **DirectX ASM**
Шейдерный ассемблер от Microsoft
- **Cg (C for Graphics)**
Шейдерный язык от nVidia для Microsoft
- **RenderMan**
Шейдерный язык от Pixar для художников
- **Gelato**
Шейдерный язык от nVidia для художников

GLSL – это высокоуровневый процедурный язык программирования, декларированный стандартами OpenGL и Vulkan

- GLSL предоставляет абстракцию от аппаратного обеспечения
- Компилятор GLSL является частью библиотеки стандарта OpenGL и генерирует оптимизированный под конкретную видеокарту код
- GLSL основан на синтаксисе C и является чисто процедурным
- Начало выполнения программы – функция `void main()`
- Спецификация языка (от 31.07.2017, версия 4.60):
<https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>

Шейдеры могут получать из основной программы произвольные наборы данных:

- «глобальных» (одинаковых для всех обрабатываемых сущностей):
uniform-переменные
- «локальных» (сцепленных с сущностью, как правило, при помощи массивов, индексы в которых соответствуют номерам сущностей):
attribute-переменные

Шейдеры более раннего этапа конвейера могут подготавливать и передавать параметры шейдерам более позднего этапа

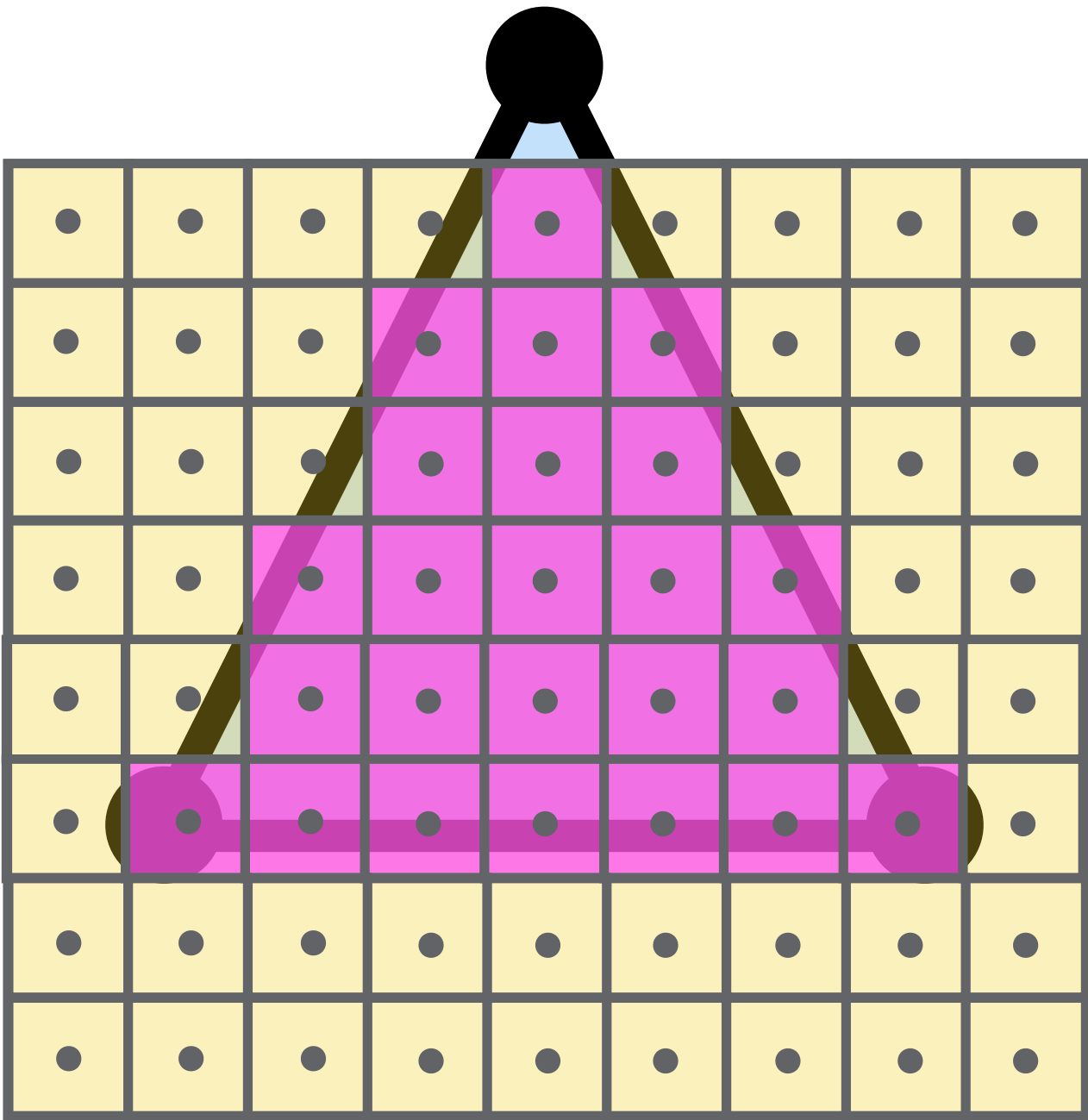
- **varying-переменные**

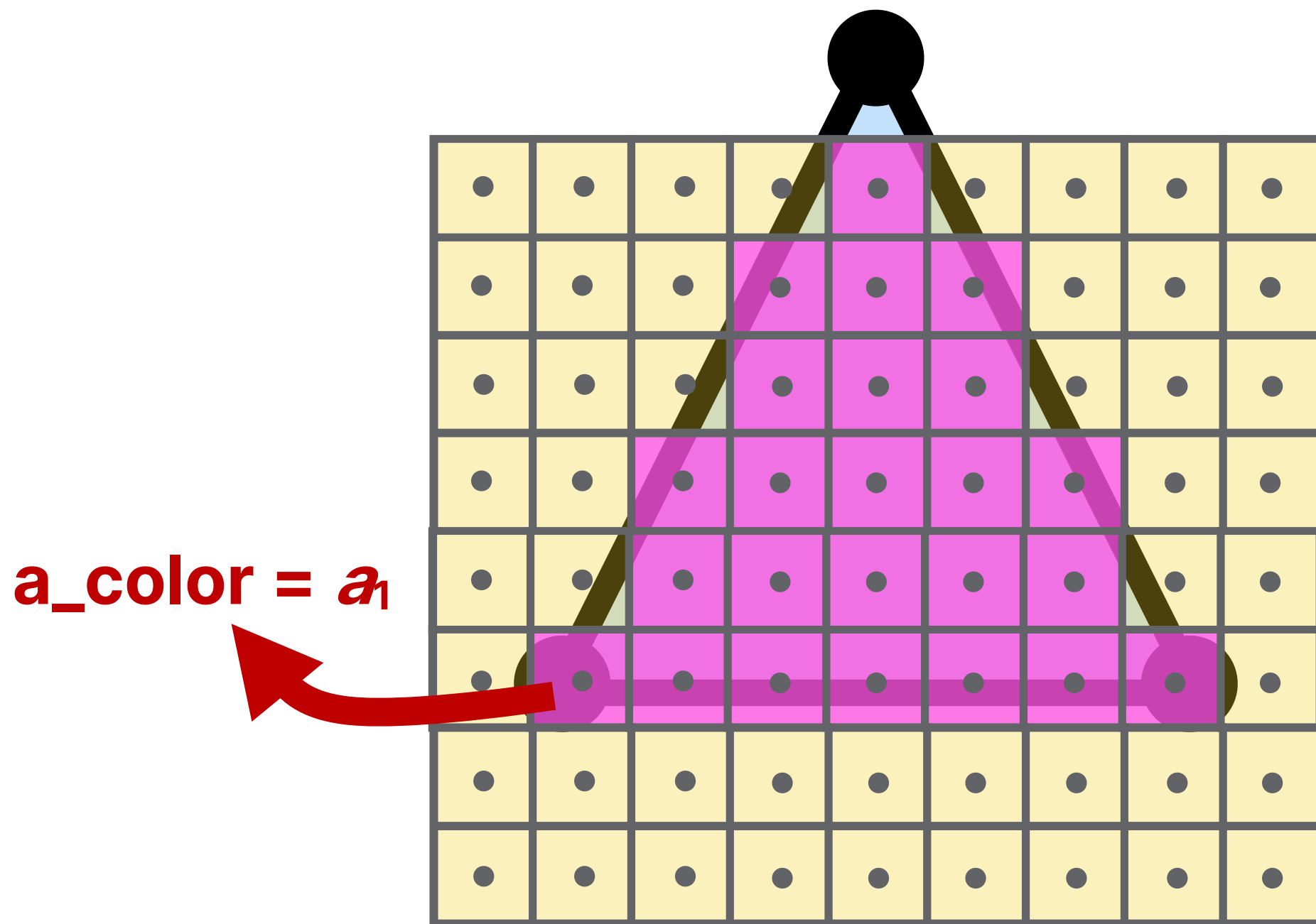
Вершинный шейдер:

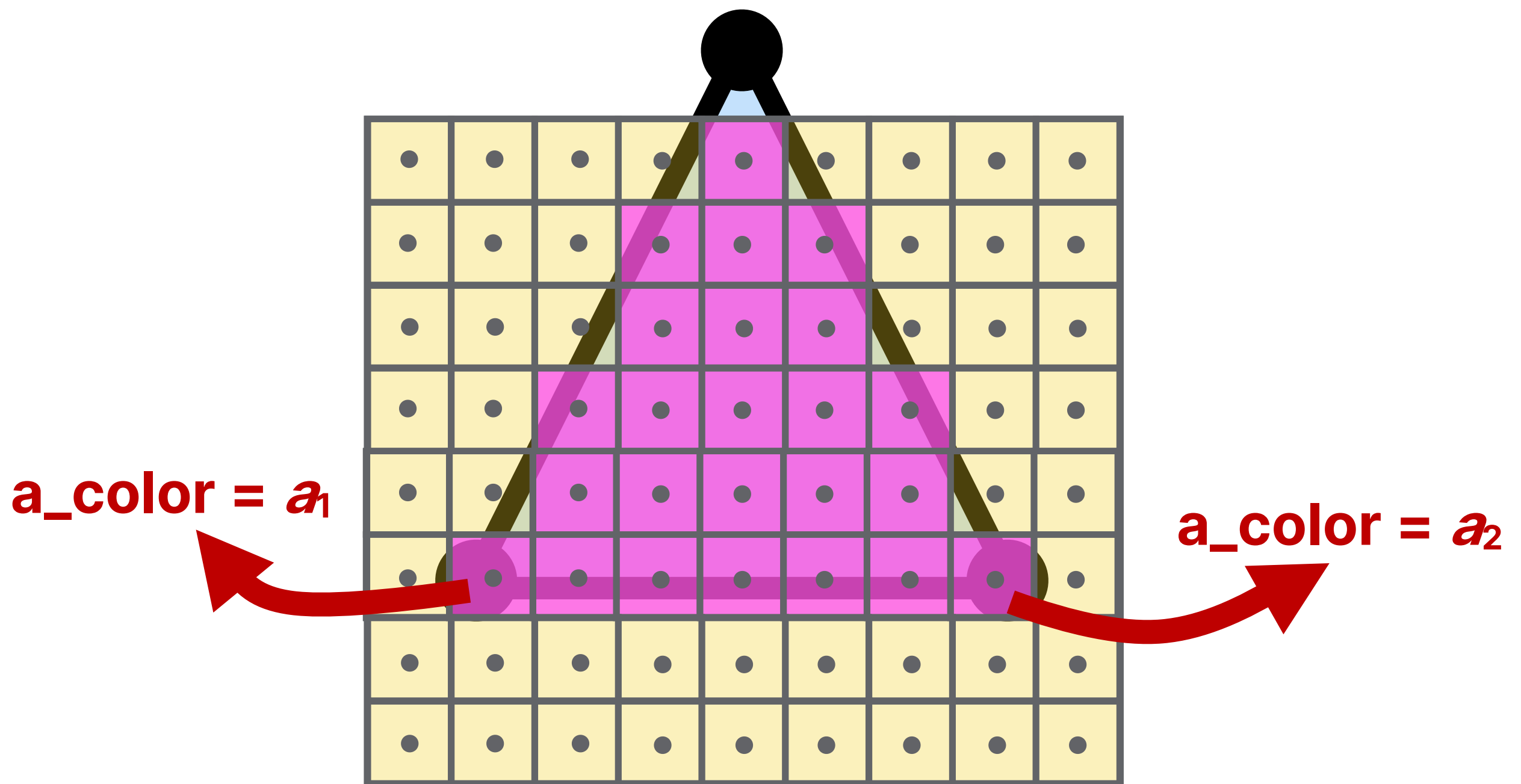
```
uniform mat4 u_MVP;  
layout(location = 0) in vec3 a_position;  
layout(location = 1) in vec3 a_color;  
out vec3 v_color;  
void main() {  
    v_color = a_color;  
    gl_Position = u_MVP * vec4(a_position, 1.0);  
}
```

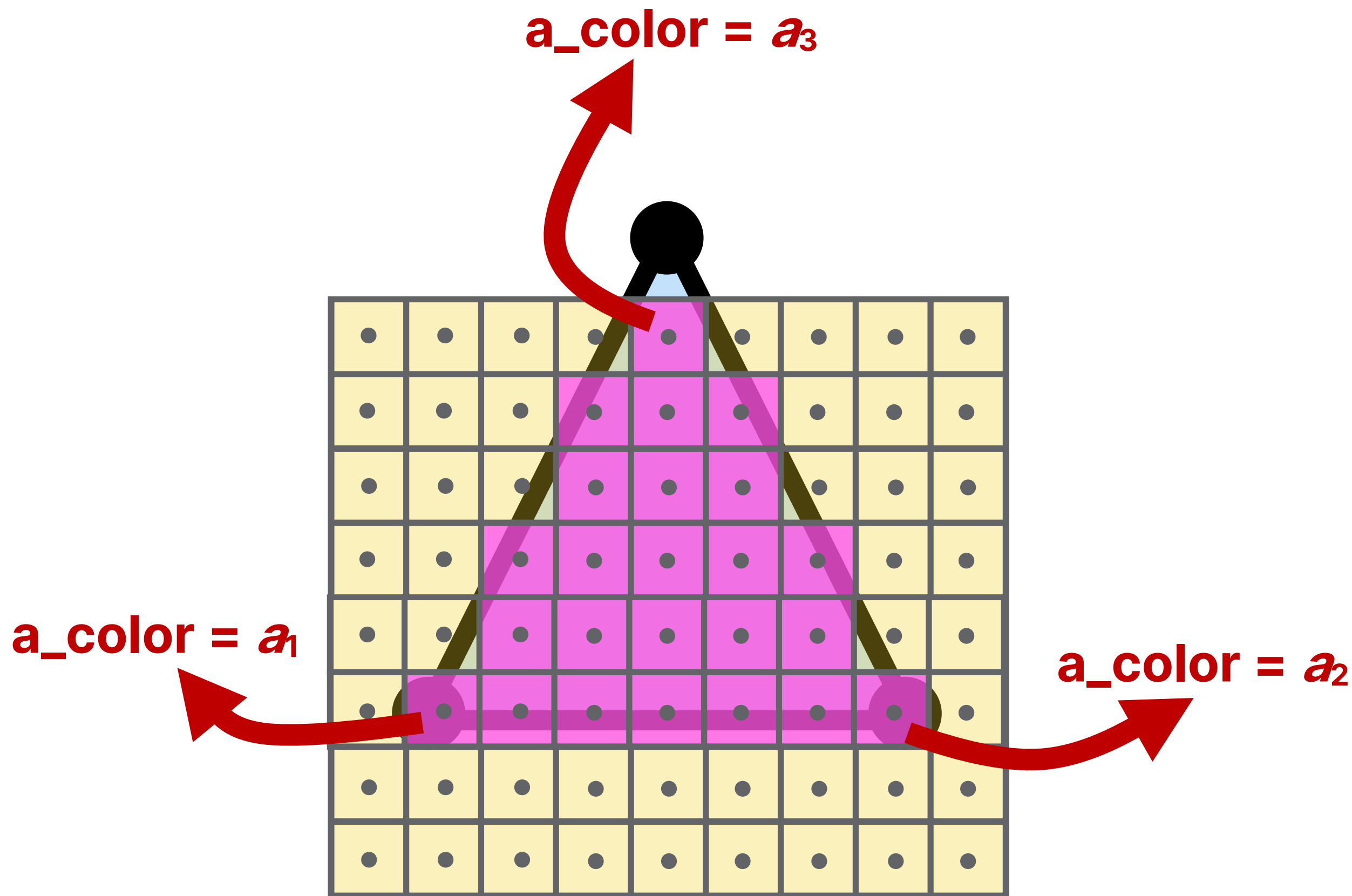
Фрагментный шейдер:

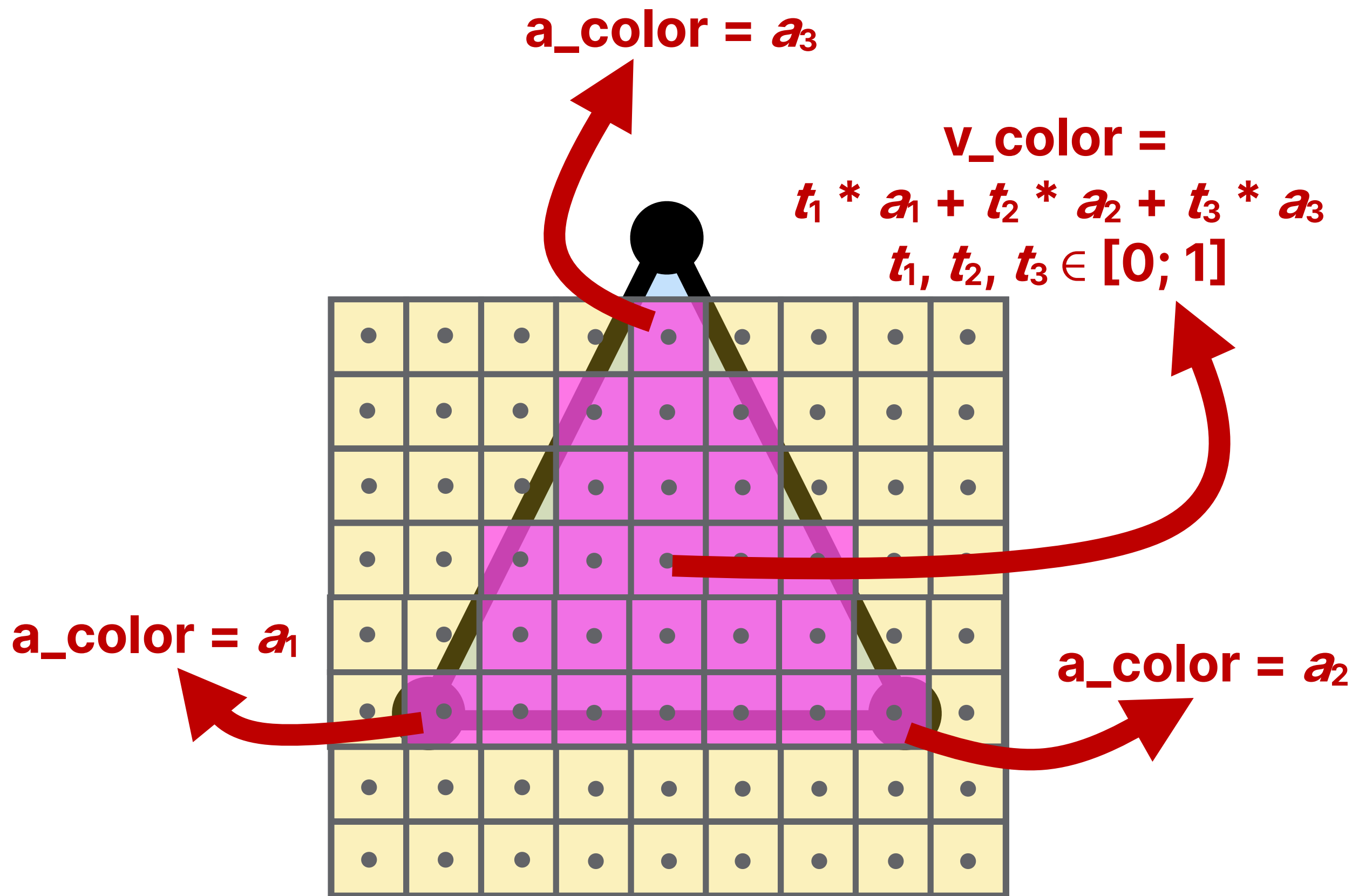
```
uniform float u_alpha;  
in vec3 v_color;  
layout(location = 0) out vec4 o_color;  
void main() {  
    o_color = vec4(v_color, u_alpha);  
}
```











Шейдерная программа – специализированная структура данных для хранения группы шейдеров разных типов с целью их совместного использования

- В самом простом случае шейдерная программа состоит из двух шейдеров – одного вершинного и одного фрагментного (программируемый конвейер в каждый момент времени нуждается минимум в двух шейдерах)
- Возможна сборка одного шейдера из нескольких частей
- В каждый момент времени активна только одна шейдерная программа
- Шейдерная программа – единственный способ использования шейдеров в OpenGL
- Порядок вызовов:

`glCreateProgram → glAttachShader → glAttachShader → glLinkProgram → glUseProgram`

1. **vec2, vec3, vec4** – вещественные вектора на 2, 3 и 4 компоненты
2. **ivec2, ivec3, ivec4** – целочисленные вектора на 2, 3 и 4 компоненты
3. **bvec2, bvec3, bvec4** – логические вектора на 2, 3 и 4 компоненты

Особенности:

- Реализована перегрузка операций сложения и вычитания векторов, причём код оптимизируется и выполняется GPU за $O(1)$
- Инициализация вектора может быть осуществлена при помощи конструкторов вида
`vec3 a = vec3(0.1, 0.2, 0.3);`
`vec4 b = vec4(a, 0.4);`
`vec2 c = vec2(a); // будут взяты первые две компоненты`
- Для доступа к компонентам можно использовать индекс:
`vec3 a; a[1] = 0.5;`
либо мнемонические поля (x, y, z, w ~ r, g, b, a ~ s, t, p, q):
`vec3 a; a.y = 0.5;`
- Существуют вспомогательные поля («коктейли»), предоставляющие доступ к любому подмножеству значений в любой последовательности:
`vec3 a, b;`
`a.xy = b.zy = vec2(0.5, 0.8);`
`vec3 c = vec3(0.9, a.xy);`
- Мнемоника полей существует лишь для пользователя, представляя собой обёртку доступа к данным

1. **mat2** – вещественная матрица 2x2
2. **mat3** – вещественная матрица 3x3
3. **mat4** – вещественная матрица 4x4

Особенности:

- Реализована перегрузка операций сложения, вычитания и умножения матриц
- Реализована перегрузка операции умножения матрицы на вектор
- Матрицы хранятся по столбцам и могут быть рассмотрены как массив векторов-столбцов
- Как правило, матрицы приходят в шейдер из основной программы и используются для произведения аффинных преобразований

Структуры

```
struct Light { vec3 position; vec3 color; };
```

Особенности:

- Структуры полностью идентичны структурам в C
- `union` и `enum` зарезервированы в качестве ключевых слов, но пока не поддерживаются

Массивы

```
float a[10];
```

```
vec4 points[5];
```

Особенности:

- Можно объявлять массивы любых типов
- Массивы являются статическими

`void` – тип для функций, не возвращающих значения

Не поддерживаются:

- 1. Указатели**
- 2. Динамическая память (кроме стека)**
- 3. Передача значений по ссылке**
- 4. Строки**
- 5. Классы**
- 6. Пережающие описания функций**
- 7. Рекурсия (как прямая, так и косвенная)**

- Перегрузка операций для векторных и матричных типов данных
- Функции над векторами:
 - **dot** – скалярное произведение
 - **cross** – векторное произведение
 - **normalize** – нормирование вектора
 - **reflect** – отражение вектора относительно вектора
 - **refract** – преломление вектора относительно вектора с коэффициентом преломления
 - **length** – длина вектора
 - **distance** – расстояние между двумя точками
 - ...

- **Функции над матрицами:**
 - **determinant** – определитель матрицы
 - **transpose** – транспонированная матрица
 - **inverse** – обратная матрица
 - ...
- **Тригонометрические функции:**
 - **sin, cos, tan** – функции
 - **asin, acos, atan** – аркфункции
 - **radians, degrees** – перевод из градусов в радианы и обратно
 - ...
- **Гиперфункции:**
 - **sinh, cosh, tanh** – функции
 - **asinh, acosh, atanh** – аркфункции
 - ...

- **Математические функции**
 - **pow** – возведение произвольную в степень
 - **exp** – экспонента
 - **log** – натуральный логарифм
 - **sqrt** – квадратный корень
 - **clamp** – ограничение значения
 - **abs** – модуль
 - **ceil, floor, round** – округление в разные стороны
 - **sign** – сигнум (знак числа)
 - **min, max** – минимум, максимум
 - ...
- ...